Okostelefon és IoT eszközök programozása

Autós bolt jegyzőkönyv

Készítette: Stremler László

Neptunkód: AQYO8L

Tartalomjegyzék

1. Bevezetés **1**

2. Felület **2**

2.1. A felület elemei 2

2.1.1. Főképernyő 2

2.1.2. Dialógusok 6

2.2. Kosár 7

3. A funkciók implementációja **10**

3.1. Projekt struktúra felépítése 10

3.2. SQLite adatbázis kapcsolat 17

3.3. Gomb műveletek 18

1. fejezet

Bevezetés

Az Okostelefon és IoT eszközök programozás órára készített féléves beadandóm egy autós bolt androidos alkalmazás implementációját mutatja be. Az alkalmazás fejlesztése során kiemelt figyelmet fordítottam a felhasználóbarát felület kialakítására, valamint az alapvető CRUD (Create, Read, Update, Delete) műveletek hatékony megvalósítására.

Az alkalmazásban lehetősége van a felhasználónak hozzáadni, módosítani és törölni autót a megjelenített listából, ami dinamikusan frissül az adatbázis tartalmának megfelelően. Az egyes autókhoz különböző attribútumok tartoznak, mint például a márka, típus, ár, szín, max sebesség, 0-ról 100-ra való gyorsulás. Emellett az alkalmazás támogatja a kosárkezelési funkciókat is: a felhasználó kosárhoz adhatja a kiválasztott autókat, majd egy külön felületen megtekintheti azokat, illetve dönthet azok megvásárlásáról vagy kosárból való eltávolításáról.

Az autókat és a kosarat egy SQLite adatbázisban tárolja az alkalmazás, amely egy alkalmazáson belüli adatbázist jelent, ami csak futási időben létezik. Ez a megoldás különösen hasznos fejlesztési és tesztelési környezetben, mivel gyors hozzáférést biztosít. Az SQLite különlegessége a többi memóriában futó adatbázissal szemben, hogy lehetőséget biztosít hagyományos SQL parancsok használatára, így az adatkezelés strukturált és jól átlátható módon valósítható meg.

Összességében a beadandó célja az volt, hogy bemutassa egy egyszerű e-kereskedelmi alkalmazás alapvető működését mobil környezetben, különös tekintettel az adatkezelésre, a felhasználói interakciókra és az alkalmazás logikai felépítésére.

2. fejezet

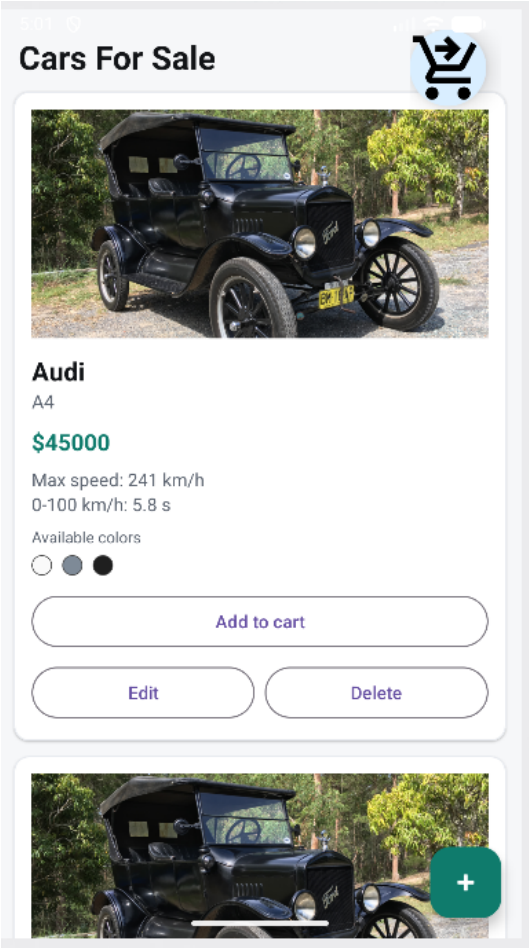
Felület

2.1. A felület elemei

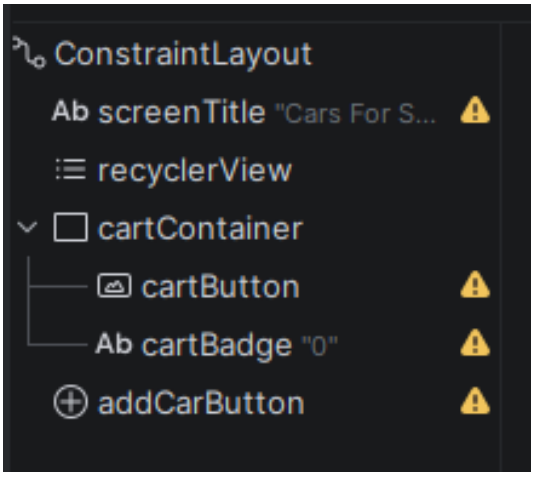
A felület több komponensből áll, amelyek együttesen valósítják meg a kinézetet. Ezen komponensek egy része újrahasznosítható komponens, amely több funkció ellátásakor is felhasználásra került.

2.1.1. Főképernyő

A főképernyő gyakorlatilag a megvásárolható autók listája. Az alkalmazás elindításakor ez a képernyő fogadja a felhasználót. Erről a felületről lehet belépni a kosárba, illetve itt lehet hozzáadni autókat.



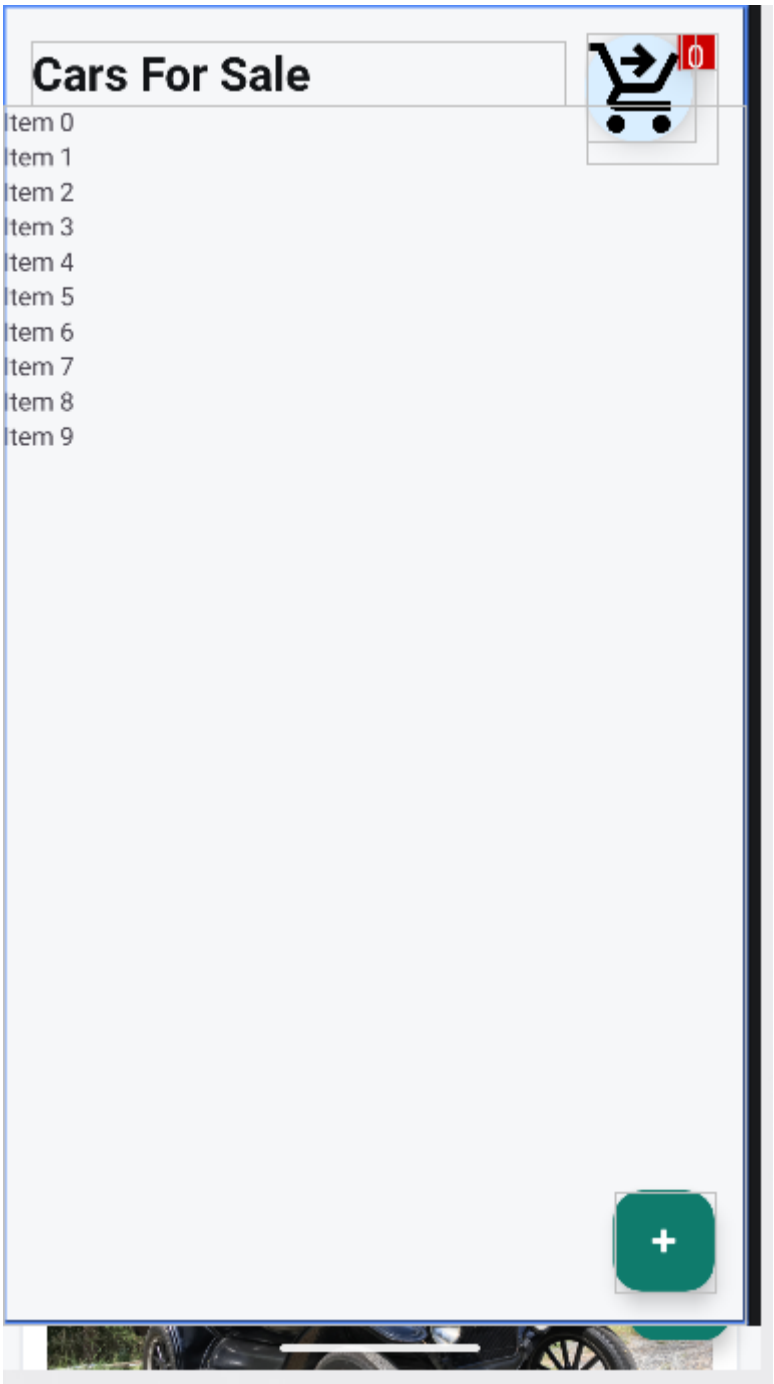
(a) Az alkalmazás főképernyője



(b) Az alkalmazás főképernyőjének felépítése

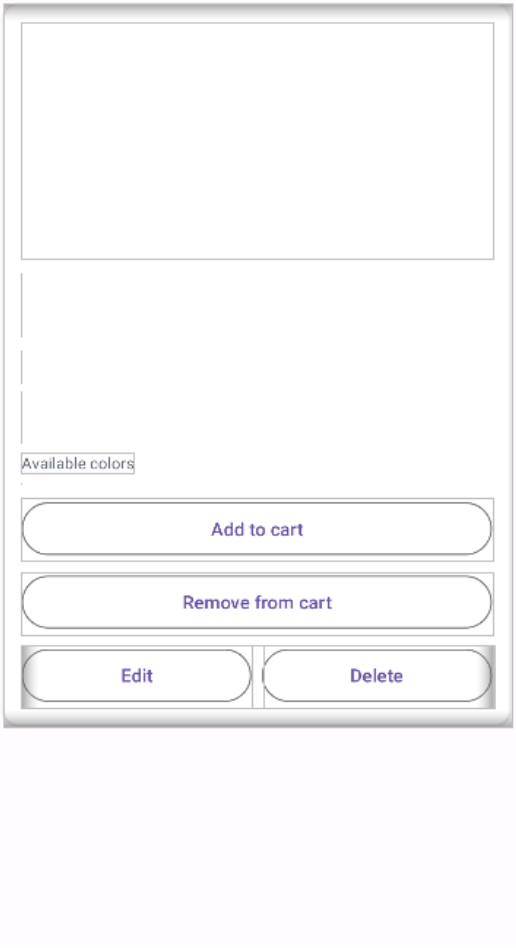
A főképernyő a következő elemekből áll:

- ConstraintLayout - Az elrendezésért felelős XML tag
- screenTitle - "Cars For Sale" felirat a képernyő tetején
- recyclerView - Az autó lista ismétléséért felel
- cartButton - Kosár gomb a cartContainer-ben
- cartBadge - Kosár tartalmát indikáló jelzó a cartContainer-ben
- addCarButton - Autó hozzáadó gomb a jobb alsó sarokban

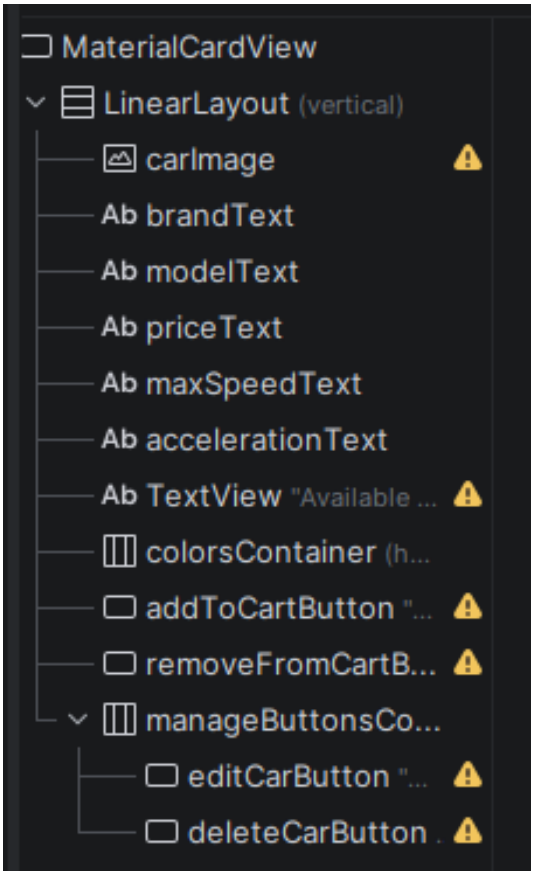


2.2. ábra. A főképernyő tervezete

A képen látható item-ek a recyclerView helyeit mutatják, amelyek egy-egy autó kártyát reprezentálnak.



(a) Az autó kártya



(b) Az autó kártya felépítése

Az autó kártya a következő elemekből áll:

- MaterialCardView - Kártyás kinézetet biztosító komponens
- LinearLayout - Lineáris elrendezésért felelős komponens
- carImage - Autó képe
- brandText - Autó márkája
- modelText - Autó típusa
- priceText - Autó ára
- maxSpeedText - Autó végsebessége
- accelerationText - Autó 0-100-ra való gyorsulása
- TextView - "Available colors" felirat
- colorsContainer - Autó színeit tartalmazó gyűjtő

- addToCartButton - Kosárhoz hozzáadó gomb (csak lista módban jelenik meg)
- removeFromCartButton - Kosárból eltávolító gomb (csak kosár módban jelenik meg)
- manageButtonsContainer - Autó módosító gombok gyűjtője
- editCarButton - Autó szerkesztő gomb (csak lista módban jelenik meg)
- deleteCarButton - Autó törlő gomb (csak lista módban jelenik meg)

A képernyő jobb felső sarkában található a kosár, amelynek indikátora jelzi hány elem van a kosárban. Ha üres a kosár, akkor az indikátor eltűnik.

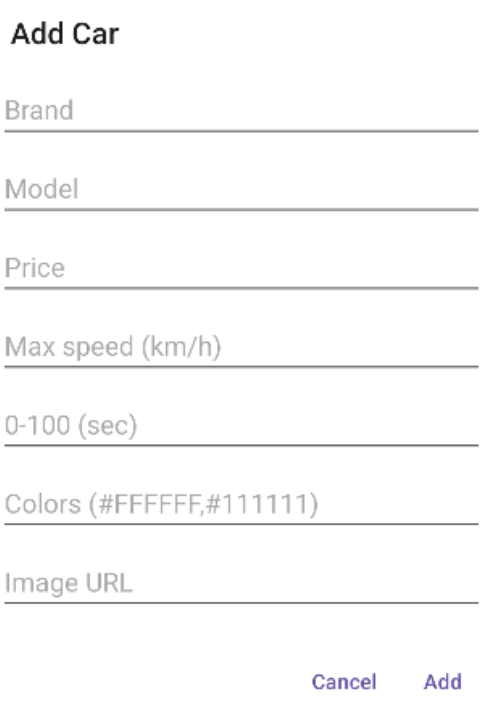
(a) Üres kosár

(b) Elemet tartalmazó kosár

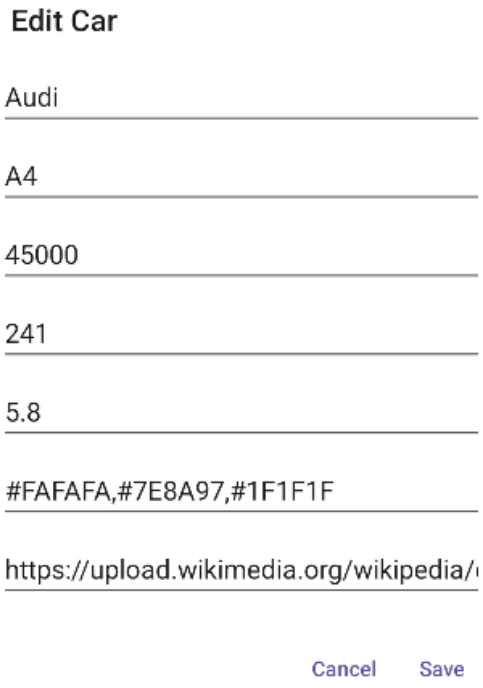
A képernyő jobb alsó sarkában található plusz gomb segítségével lehet autót hozzáadni a listához.

2.1.2. Dialógusok

Az alkalmazásban több helyen is előfordulnak dialógusok: autó hozzáadás, autó szerkesztés, törlés megerősítés. A hozzáadó dialógus megegyezik a szerkesztő dialógussal, mivel egy újrafelhasználható komponensként van használva. A törlés megerősítés dialógus egy beépített, AlerDialog típusú objektum.



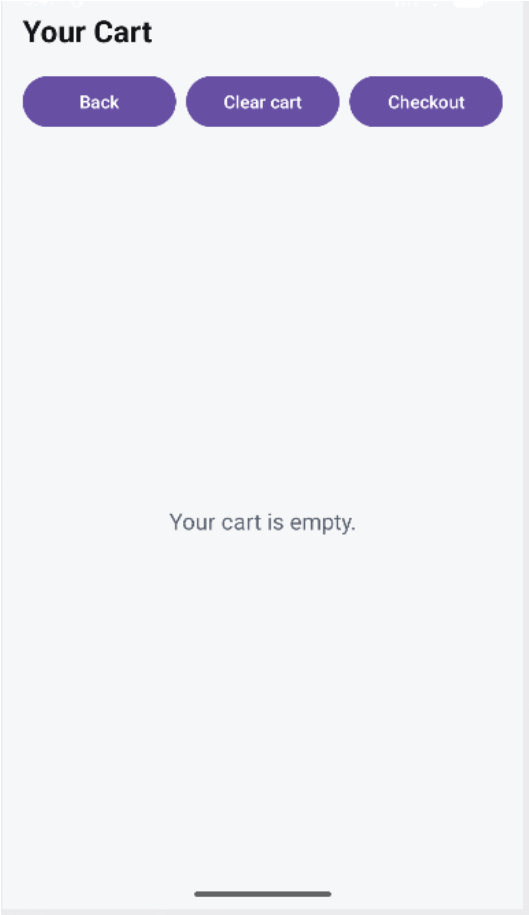
(a) Hozzáadó dialógus



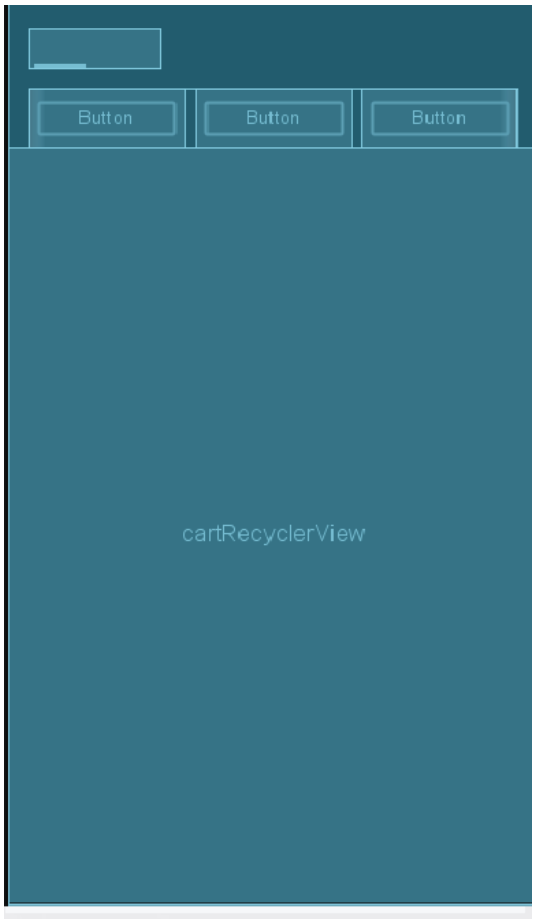
(b) Szerkesztő dialógus

2.2. Kosár

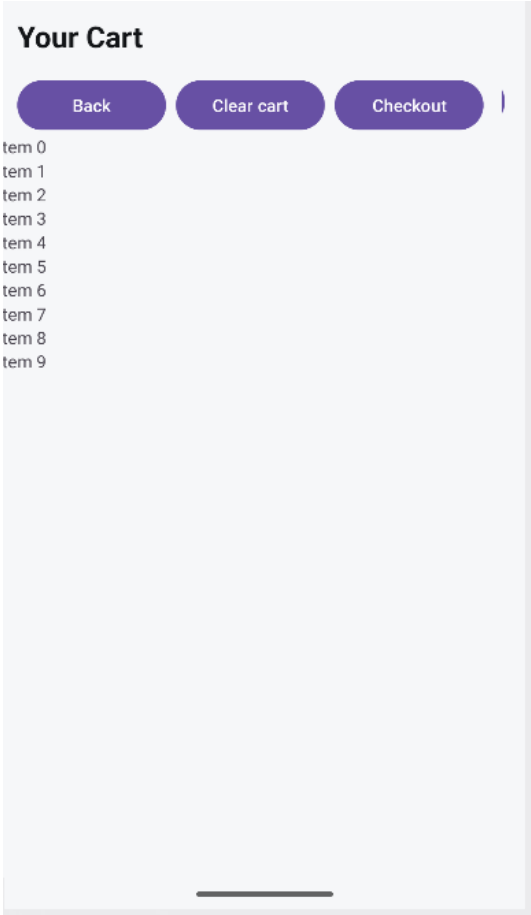
A kosár képernyő lényegében az autó kártya újrafelhasználható komponenst használja listaként, kosár módban, ezáltal csak az eltávolítás gomb jelenik meg a kártyán. Ezenkívül a komponens még három gombból áll: Back, Clear cart, Checkout. A Back gomb visszalép a főoldalra, a Clear cart gomb törli a kosár tartalmát és a Checkout gomb pedig "megvásárolja" a kosárban lévő termékeket (gyakorlatilag csak törli a listát).



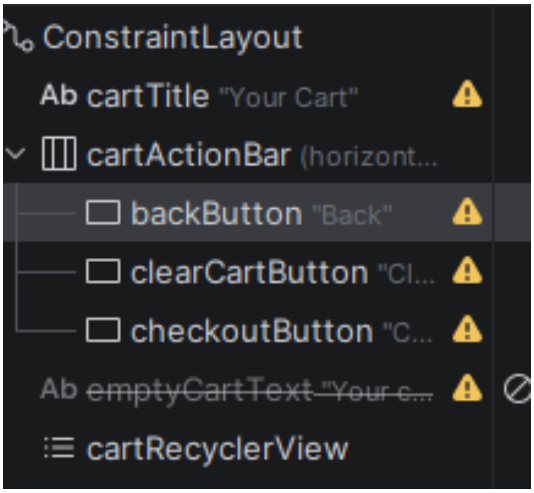
(a) A kosár képernyő



(b) A kosár képernyő terve



(a) A kosár képernyő felépítése



(b) Kosár képernyő elemei

A kosár képernyő az alábbi elemekből áll:

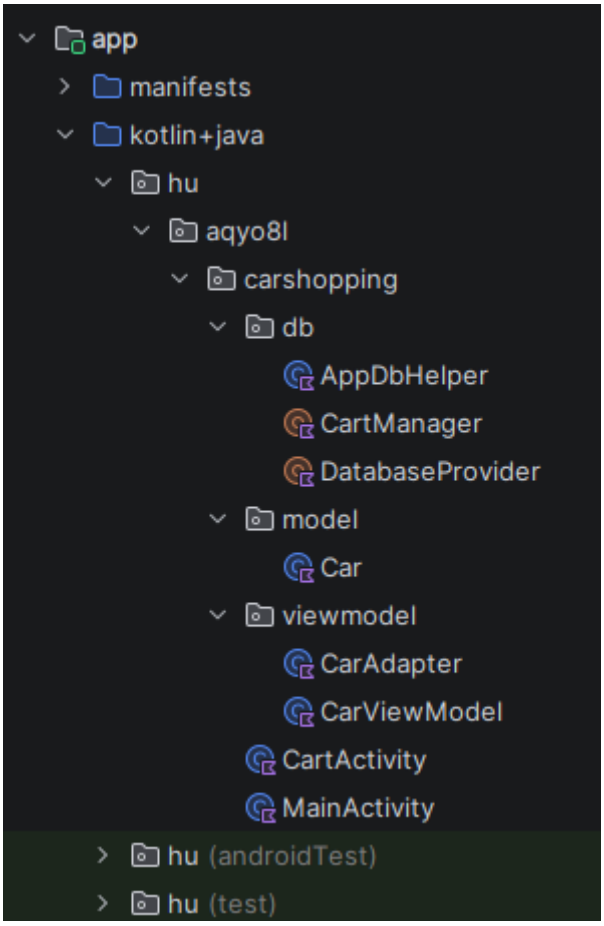
- ConstraintLayout - Elrendezésért felelős elem
- cartTitle - "Your Cart" felirat
- cartActionBar - Gombokat tartalmazó gyűjtő
- backButton - Vissza gomb
- clearCartButton - Kosár ürítő gomb
- checkoutButton - Vásárlás gomb
- emptyCartText - "Your cart is empty" felirat, alapból letiltva
- cartRecyclerView - A kosárban lévő autók listája, kosár módban (csak az eltávolítás gomb látszik)

3. fejezet

A funkciók implementációja

3.1. Projekt struktúra felépítése

A projekt felépítése négy fő részt tartalmaz: adatbázis konfigurációhoz, csatlakozáshoz köthető osztályok, modell osztály, nézetmodell osztály, activity-k.



3.1. ábra. A projekt szerkezete

Az adatbázis osztályok a db package-ben kerültek elhelyezésre. Három osztály ta-

lálható ebben a package-ben:

- AppDbHelper - Adatbázis kapcsolat létrehozásáért felelős osztály, ez az osztály hozza létre a táblákat, hajtja végre a beszúró, lekérdező, törlő, módosító query-t. Implementálja a SQLiteOpenHelper osztályt, amely egy gyári SQLite adatbázis osztály.

```
3 Usages
class AppDbHelper(context: Context) : SQLiteOpenHelper(context, DB_NAME, factory = null, DB_VERSION) {

    1 Usage
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(
            """
            CREATE TABLE cars (
                id INTEGER PRIMARY KEY,
                brand TEXT NOT NULL,
                model TEXT NOT NULL,
                price INTEGER NOT NULL,
                img_url TEXT NOT NULL,
                max_speed_kmh INTEGER NOT NULL,
                zero_to_hundred_sec REAL NOT NULL,
                available_colors TEXT NOT NULL
            )
            """.trimIndent()
        )

        db.execSQL(
            """
            CREATE TABLE cart_items (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                car_id INTEGER NOT NULL,
                FOREIGN KEY(car_id) REFERENCES cars(id)
            )
            """.trimIndent()
        )
```

3.2. ábra. Példa az AppDbHelper-ben szereplő függvényekre: Az adatbázis táblákat létrehozó függvény

- CartManager - A kosár tartalmáért felelős osztály. Az adatbázis műveleteket a DatabaseProvider-en keresztül végzi.

```
10 Usages
object CartManager {
    1 Usage
    fun addToCart(car: Car) {
        DatabaseProvider.db().addToCart( carId = car.id)
    }

    1 Usage
    fun getCartItems(): List<Car> = DatabaseProvider.db().getCartItems()

    2 Usages
    fun getCartCount(): Int = DatabaseProvider.db().getCartCount()

    1 Usage
    fun removeOneFromCart(car: Car) {
        DatabaseProvider.db().removeOneFromCart( carId = car.id)
    }

    2 Usages
    fun clearCart() {
        DatabaseProvider.db().clearCart()
    }
}
```

3.3. ábra. CartManager osztály

- DatabaseProvider - AppDbHelper osztályt wrappelő osztály, amely egy köztes réteget biztosít az adatbázis kezelő (AppDbHelper) osztály és egy alkalmazás osztály (CartManager, CarViewModel, MainActivity) között.

```
1   package hu.aqyo8l.carshopping.db
2
3   import android.content.Context
4
    15 Usages
5   object DatabaseProvider {
        4 Usages
6       @Volatile
7       private var dbHelper: AppDbHelper? = null
8
        2 Usages
9       fun init(context: Context) {
10          if (dbHelper == null) {
11              synchronized( lock = this) {
12                  if (dbHelper == null) {
13                      dbHelper = AppDbHelper(context.applicationContext)
14                  }
15              }
16          }
17      }
18
        10 Usages
19      fun db(): AppDbHelper {
20          return checkNotNull( value = dbHelper) { "DatabaseProvider is not initialized." }
21      }
22  }
23
```

3.4. ábra. A DatabaseProvider osztály

A model package-ben az autót leíró Car adatosztály került elhelyezésre.

```
data class Car(
    val id: Int,
    val brand: String,
    val model: String,
    val price: Int,
    val imgUrl: String,
    val maxSpeedKmh: Int,
    val zeroToHundredSec: Double,
    val availableColors: List<String>
)
```

3.5. ábra. Car osztály

A viewmodel package-ben két osztály kapott helyet:

- CarAdapter - Ez az osztály felelős a képernyőn megjelenő autó kártyák adattal való feltöltéséért.

```
override fun onBindViewHolder(holder: CarViewHolder, position: Int) {
    val car = cars[position]
    holder.brand.text = car.brand
    holder.model.text = car.model
    holder.price.text = "$${car.price}"
    holder.maxSpeed.text = "Max speed: ${car.maxSpeedKmh} km/h"
    holder.acceleration.text = String.format(
        Locale.US,
        format = "0-100 km/h: %.1f s",
        ...args = car.zeroToHundredSec
    )

    Glide.with(holder.itemView.context)
        .load( string = car.imgUrl)
        .into( view = holder.image)

    renderColorDots(holder, car)

    if (showAddButton) {
        holder.addButton.visibility = View.VISIBLE
        holder.addButton.setOnClickListener {
            CartManager.addToCart(car)
            onAddToCart?.invoke()
        }
    } else {
        holder.addButton.visibility = View.GONE
    }

    if (showRemoveButton) {
        holder.removeButton.visibility = View.VISIBLE
        holder.removeButton.setOnClickListener {
            CartManager.removeOneFromCart(car)
            onRemoveFromCart?.invoke()
        }
    } else {
        holder.removeButton.visibility = View.GONE
    }
```

3.6. ábra. Példa az adatfeltöltésre a CarAdapter osztályból

- CarViewModel - Ez az osztály tartalmazza a megvásárolható autók listáját, valamint felelős azok adatbázisból való betöltésének meghívásáért (a DatabaseProvider-en keresztül).

```
2 Usages
class CarViewModel : ViewModel() {
    2 Usages
    private val _cars = MutableLiveData<List<Car>>()
    1 Usage
    val cars: LiveData<List<Car>> = _cars

    3 Usages
    fun loadCars() {
        _cars.value = DatabaseProvider.db().getCars()
    }
}
```

3.7. ábra. CarViewModel osztály

Az alkalmazás főoldalát a MainActivity írja le és vezérli, míg a kosár oldalt a CartActivity osztály. Ezek tartalmazzák többek között a gombra kattintáskor bekövetkező eseményeket, ezek az osztályok felelnek bármilyen felületi frissítéskor bekövetkező esemény kezeléséért.

Az alkalmazás indulásakor a MainActivity-ben lévő onCreate() függvény fut le elsőnek. Ez a függvény inicializálja a DatabaseProvider osztályt, amely felállítja az adatbázis kapcsolatot. Beállításra kerülnek a gombok, először lekérdezi a felületről az azonosítójuk segítségével az adott elemet, majd értékül adja egy változónak. (Pl.: cartButton = findViewById(R.id.cartButton) ez a sor a cartButton változónak értékül adja a cartButton azonosítójú View-t. Példányosításra került a CarAdapter osztály, amely felelős a megjelenő autók paramétereinek helyes feltöltéséért. Ezenkívül fontos még megemlíteni, hogy ebben a függvényben kerül beállításra minden gomb kattintáskor végrehajtandó függvénye (a képen ez a 67. és 70. sorban valósul meg).

```
26      class MainActivity : AppCompatActivity() {
33          override fun onCreate(savedInstanceState: Bundle?) {
34              super.onCreate(savedInstanceState)
35              setContentView(R.layout.activity_main)
36              DatabaseProvider.init(applicationContext)
37
38              val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)
39              cartButton = findViewById(R.id.cartButton)
40              val addCarButton = findViewById<ImageView>(R.id.addCarButton)
41              cartBadge = findViewById(R.id.cartBadge)
42              val cartContainer = findViewById<View>(R.id.cartContainer)
43
44              adapter = CarAdapter(
45                  cars = emptyList(),
46                  showAddButton = true,
47                  showManageButtons = true,
48                  onAddToCart = {
49                      updateBadge(cartBadge)
50                      playCartAddAnimation()
51                  },
52                  onEditCar = { showCarFormDialog( car = it) },
53                  onDeleteCar = { deleteCar(it) }
54              )
55              recyclerView.layoutManager = LinearLayoutManager( context = this)
56              recyclerView.adapter = adapter
57
58              viewModel.cars.observe( owner = this) {
59                  adapter.updateData( newCars = it)
60              }
61              viewModel.loadCars()
62
63              cartContainer.bringToFront()
64
65              updateBadge(cartBadge)
66
67              cartButton.setOnClickListener {
68                  startActivity(Intent( packageContext = this,  cls = CartActivity::class.java))
69              }
70              addCarButton.setOnClickListener {
71                  showCarFormDialog()
72              }
73          }
```

3.8. ábra. Az alkalmazás indulásakor lefutó függvény, a MainActivity-ben lévő onCreate()

Többek között az autó törlést is a MainActivity hajtja végre, ezt a deleteCar függvény végzi, amely a törlő gomb onClickListener paraméterén van értékként beállítva. A függvény elvár egy autót paraméterként, majd előállít egy gyári, AlertDialog típusú dialógust, amely megkérdezi a felhasználót, hogy biztos szeretné-e törölni a kiválasztott autót. A setPositiveButton függvényben található a Delete válasz esetén végrehajtandó kód, amely nem mást csinál, mint a DatabaseProvider-en keresztül elvégzi az autó eltávolítását az adatbázisból, majd újratölti a listát, frissíti a kosár elemszámát és küld egy visszajelző üzenetet. Nemleges válasz esetén a dialógus minden további nélkül

bezárásra kerül.

```
1 Usage
private fun deleteCar(car: Car) {
    AlertDialog.Builder( context = this)
        .setTitle("Delete Car")
        .setMessage("Delete ${car.brand} ${car.model} from sale?")
        .setPositiveButton( text = "Delete") { _, _ ->
            DatabaseProvider.db().deleteCar( carId = car.id)
            viewModel.loadCars()
            updateBadge(cartBadge)
            Toast.makeText( context = this,  text = "Car deleted.",  duration = Toast.LENGTH_SHORT).show()
        }
        .setNegativeButton( text = "Cancel",  listener = null)
        .show()
}
```

3.9. ábra. A MainActivity-ben lévő deleteCar(car: Car) függvény

A CartActivity onCreate() függvénye akkor fut le, amikor a felhasználó belép a kosárba. Ezen függvényen belül kerülnek beállításra a felületi elemek, valamint a gombok kattintáskori eseményei.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_cart)
    DatabaseProvider.init(applicationContext)

    val recyclerView = findViewById<RecyclerView>(R.id.cartRecyclerView)
    val backButton = findViewById<Button>(R.id.backButton)
    val clearCartButton = findViewById<Button>(R.id.clearCartButton)
    val checkoutButton = findViewById<Button>(R.id.checkoutButton)
    emptyCartText = findViewById(R.id.emptyCartText)
    adapter = CarAdapter(
        cars = emptyList(),
        showAddButton = false,
        showRemoveButton = true,
        onRemoveFromCart = { refreshCart() }
    )

    recyclerView.layoutManager = LinearLayoutManager( context = this)
    recyclerView.adapter = adapter

    clearCartButton.setOnClickListener {
        CartManager.clearCart()
        refreshCart()
        Toast.makeText( context = this,  text = "Cart cleared.",  duration = Toast.LENGTH_SHORT).show()
    }

    checkoutButton.setOnClickListener {
        if (CartManager.getCartCount() == 0) {
            Toast.makeText( context = this,  text = "Cart is empty.",  duration = Toast.LENGTH_SHORT).show()
        } else {
            CartManager.clearCart()
            refreshCart()
            Toast.makeText( context = this,  text = "Payment successful. Order placed!",  duration = Toast.LENGTH_LONG).show()
        }
    }

    backButton.setOnClickListener {
        finish()
    }
```

3.10. ábra. A CartActivity onCreate() függvénye

3.2. SQLite adatbázis kapcsolat

Az adatbázis kapcsolatért az AppDbHelper osztály felel, ebben az osztályban történik a táblák létrehozása, a rekordok beszúrása, kiolvasása, szerkesztése, törlése.

Az SQLiteOpenHelper beépített osztályt használja, amely egy alkalmazáson belül létező adatbázist hoz létre az alkalmazás számára.

Az osztály példányosításakor (alkalmazás elindításakor) létrehozza az adatbázistáblákat az onCreate() függvényben. A létrehozást követően feltölti alap adatokkal a cars táblát. Ez az adathalmaz jelenik meg első indításkor a felhasználónak.

```
1 Usage
override fun onCreate(db: SQLiteDatabase) {
    db.execSQL(
        """
        CREATE TABLE cars (
            id INTEGER PRIMARY KEY,
            brand TEXT NOT NULL,
            model TEXT NOT NULL,
            price INTEGER NOT NULL,
            img_url TEXT NOT NULL,
            max_speed_kmh INTEGER NOT NULL,
            zero_to_hundred_sec REAL NOT NULL,
            available_colors TEXT NOT NULL
        )
        """.trimIndent()
    )

    db.execSQL(
        """
        CREATE TABLE cart_items (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            car_id INTEGER NOT NULL,
            FOREIGN KEY(car_id) REFERENCES cars(id)
        )
        """.trimIndent()
    )

    seedCars(db)
}
```

3.11. ábra. Az AppDbHelper onCreate() függvénye

```
1 Usage
private fun seedCars(db: SQLiteDatabase) {
    val sampleCars = listOf(
        Car( id = 1,  brand = "Toyota",  model = "Corolla",  price = 20000,  imgUrl = SAMPLE_IMAGE_URL,  maxSpeedKmh = 210,  zeroToHundredSec = 8.2,  availableColors = listOf("#FFFFFF", "#
        Car( id = 2,  brand = "BMW",  model = "M3",  price = 70000,  imgUrl = SAMPLE_IMAGE_URL,  maxSpeedKmh = 250,  zeroToHundredSec = 3.9,  availableColors = listOf("#0A3D91", "#111111",
        Car( id = 3,  brand = "Audi",  model = "A4",  price = 45000,  imgUrl = SAMPLE_IMAGE_URL,  maxSpeedKmh = 241,  zeroToHundredSec = 5.8,  availableColors = listOf("#FAFAFA", "#7E8A97"
        Car( id = 4,  brand = "Tesla",  model = "Model 3",  price = 50000,  imgUrl = SAMPLE_IMAGE_URL,  maxSpeedKmh = 225,  zeroToHundredSec = 4.4,  availableColors = listOf("#F5F5F5", "#1
    )

    sampleCars.forEach { car ->
        val values = ContentValues().apply {
            put("id", car.id)
            put("brand", car.brand)
            put("model", car.model)
            put("price", car.price)
            put("img_url", car.imgUrl)
            put("max_speed_kmh", car.maxSpeedKmh)
            put("zero_to_hundred_sec", car.zeroToHundredSec)
            put("available_colors", encodeColors(car.availableColors))
        }
        db.insert( table = "cars",  nullColumnHack = null, values)
    }
}
```

3.12. ábra. Alap adathalmazt létrehozó függvény

3.3. Gomb műveletek

A gombra kattintáskor végrehajtandó műveletek definiálását az Acitvity-kben végeztem el.

```
clearCartButton.setOnClickListener {
    CartManager.clearCart()
    refreshCart()
    Toast.makeText( context = this,  text = "Cart cleared.",  duration = Toast.LENGTH_SHORT).show()
}
```

3.13. ábra. Példa egy gomb művelet beállításra, a kosár törlő gomb művelete